

Blocco 9 - Architettura dati containerizzata

DBMS, collector simulato e modello relazionale per dashboard

Relational Databases & SQL

30 min teoria + 90 min pratica

1. Il problema architetturale

Non basta scrivere una query: dobbiamo capire dove nasce il dato, dove viene conservato e chi lo consuma.

Obiettivo del blocco

Problema

Vogliamo costruire un modello replicabile per un piccolo sistema dati: un processo raccoglie ticket, un DBMS li conserva e Metabase li visualizza tramite query SQL.

Idea guida

Il DBMS non è solo un contenitore: è il punto in cui il dato viene vincolato, controllato, trasformato e reso interrogabile.

Scenario: ticket di assistenza

- ▶ arrivano ticket da portale, app tecnici e API partner;
- ▶ ogni ticket ha data apertura, priorità, categoria, regione, canale e stato;
- ▶ alcuni ticket vengono chiusi, altri restano aperti;
- ▶ la direzione vuole una dashboard: volumi, backlog, SLA, categorie critiche;
- ▶ gli studenti devono poter riprodurre tutto con Docker.

Architettura minima replicabile

Componente	Responsabilità
ticket-collector	Simula l'arrivo periodico di nuovi ticket.
postgres	Conserva raw data, tabelle curate, vincoli e viste SQL.
metabase	Legge PostgreSQL e costruisce card e dashboard.
docker compose	Descrive rete, porte, volumi e dipendenze.

- ❶ il collector produce o riceve un evento ticket;
- ❷ l'evento viene salvato in una tabella raw;
- ❸ il DBMS normalizza il contenuto in tabelle relazionali;
- ❹ viste SQL espongono un contratto stabile per la dashboard;
- ❺ Metabase esegue query SQL e visualizza risultati.

Perché partire dall'architettura

- ▶ separa chi produce il dato da chi lo analizza;
- ▶ rende espliciti confini e responsabilità;
- ▶ permette di ragionare su persistenza, qualità e aggiornamento;
- ▶ rende l'ambiente ripetibile: stessa rete, stessi servizi, stessi script;
- ▶ trasforma SQL da esercizio isolato a linguaggio operativo del sistema.

Analogia: una linea di produzione

Casi e responsabilità

Il collector è il punto di ingresso, come un nastro che porta pezzi grezzi. Il DBMS è la stazione di controllo qualità: accetta, rifiuta, organizza e mette etichette. La dashboard è il cruscotto che mostra come procede la linea.

2. Il DBMS nell'architettura

Il database è il contratto tra raccolta dati e consumo analitico.

Cosa deve garantire PostgreSQL

- ▶ **persistenza**: il dato resta oltre il ciclo di vita del container;
- ▶ **integrità**: vincoli, chiavi e controlli impediscono stati impossibili;
- ▶ **interrogabilità**: SQL rende il dato leggibile da persone e strumenti;
- ▶ **semantica**: viste e nomi rendono condivisa l'interpretazione;
- ▶ **osservabilità**: query di controllo mostrano qualità e completezza.

Raw e curated

	Raw	Curated
Forma	payload vicino alla sorgente	colonne tipizzate e vincolate
Uso	audit, debug, rielaborazione	query, viste, dashboard
Errore tipico	usarlo direttamente in dashboard	perdere traccia dell'origine

Schema logico del caso ticketing

- ▶ `ticket_sources`: anagrafica delle sorgenti;
- ▶ `support_tickets_raw`: eventi grezzi ricevuti dal collector;
- ▶ `support_tickets`: ticket normalizzati e interrogabili;
- ▶ `support_ticket_events`: storia degli eventi sul ticket;
- ▶ `dashboard_*`: viste orientate alla dashboard.

Esempio DDL: tabella ticket curata

```
CREATE TABLE support_tickets (  
    ticket_id bigserial PRIMARY KEY,  
    source_id smallint NOT NULL REFERENCES ticket_sources(source_id),  
    external_ticket_id text NOT NULL,  
    opened_at timestamp NOT NULL,  
    closed_at timestamp,  
    status text NOT NULL CHECK (status IN (  
        'open', 'assigned', 'waiting_customer', 'resolved', 'closed'  
    )),  
    priority text NOT NULL CHECK (priority IN ('low', 'medium', 'high', 'urgent')),  
    category text NOT NULL,  
    region text NOT NULL,  
    sla_due_at timestamp NOT NULL,  
    UNIQUE (source_id, external_ticket_id),  
    CHECK (closed_at IS NULL OR closed_at >= opened_at)  
);
```

Cosa comunica questo DDL

- ▶ ogni ticket ha un identificativo interno stabile;
- ▶ il ticket conserva anche l'identificativo della sorgente esterna;
- ▶ stato e priorità non sono testo libero;
- ▶ un ticket non può chiudersi prima dell'apertura;
- ▶ la stessa sorgente non può inviare due volte lo stesso ticket.

Contratto per la dashboard

- ▶ una dashboard non dovrebbe conoscere ogni dettaglio delle tabelle fisiche;
- ▶ può invece interrogare viste con nomi chiari e granularità dichiarata;
- ▶ una vista diventa un contratto: colonne, filtri e significato;
- ▶ se cambia la tabella interna, la vista può restare stabile;
- ▶ questo riduce la fragilità delle query in Metabase.

3. CTE e viste nel modello architetturale

CTE e viste non sono solo sintassi: sono strumenti per costruire contratti dati leggibili.

- ▶ isolare il passaggio raw → curated;
- ▶ dare nome a filtri e trasformazioni;
- ▶ testare passaggi intermedi;
- ▶ evitare query monolitiche nella dashboard;
- ▶ preparare il codice che diventerà una vista.

Esempio: vista base per dashboard

```
CREATE OR REPLACE VIEW dashboard_ticket_base AS
SELECT t.ticket_id,
       s.source_code,
       t.external_ticket_id,
       t.opened_at,
       t.closed_at,
       t.status,
       t.priority,
       t.category,
       t.region,
       t.customer_segment,
       t.channel,
       round(EXTRACT(EPOCH FROM (t.closed_at - t.opened_at)) / 3600.0, 2)
         AS resolution_hours,
       (COALESCE(t.closed_at, TIMESTAMP '2026-05-13 09:00:00') > t.sla_due_at)
         AS sla_breached
FROM support_tickets t
JOIN ticket_sources s ON s.source_id = t.source_id;
```

Perché questa vista è utile

- ▶ espone una riga per ticket;
- ▶ aggiunge il codice sorgente leggibile;
- ▶ calcola una metrica tecnica: tempo di risoluzione;
- ▶ calcola una metrica business: violazione SLA;
- ▶ diventa il punto di partenza per molte card Metabase.

Regola pratica

Prima di scrivere o usare una vista chiedersi sempre: “una riga cosa rappresenta?”.

- ▶ `dashboard_ticket_base`: una riga = un ticket;
- ▶ `dashboard_daily_flow`: una riga = un giorno;
- ▶ `dashboard_priority_status`: una riga = priorità e stato.

Esempio: flusso giornaliero

```
WITH days AS (  
    SELECT generate_series(DATE '2026-05-01',  
                           DATE '2026-05-13',  
                           interval '1 day')::date AS day  
) , opened AS (  
    SELECT opened_at::date AS day, count(*) AS opened_tickets  
    FROM support_tickets  
    GROUP BY opened_at::date  
) , resolved AS (  
    SELECT closed_at::date AS day, count(*) AS resolved_tickets  
    FROM support_tickets  
    WHERE closed_at IS NOT NULL  
    GROUP BY closed_at::date  
)  
SELECT d.day,  
       COALESCE(o.opened_tickets, 0) AS opened_tickets,  
       COALESCE(r.resolved_tickets, 0) AS resolved_tickets  
FROM days d  
LEFT JOIN opened o ON o.day = d.day  
LEFT JOIN resolved r ON r.day = d.day;
```

- ▶ collegare Metabase direttamente al payload raw per fare KPI;
- ▶ mescolare nello stesso oggetto ingestione, pulizia e dashboard;
- ▶ creare viste senza descrivere la granularità;
- ▶ nascondere filtri business importanti dentro nomi generici;
- ▶ dimenticare chiavi, vincoli e controlli perché “è solo un laboratorio”.

4. Containerizzazione

Docker Compose diventa la descrizione ripetibile dell'ambiente didattico.

Perché containerizzare

- ▶ ogni studente avvia gli stessi servizi;
- ▶ le dipendenze non vengono installate direttamente sul sistema operativo;
- ▶ porte e volumi sono documentati;
- ▶ il DBMS è raggiungibile sia da `psql` sia da Metabase;
- ▶ il collector può essere acceso, spento e osservato.

Estratto Compose

```
services:
  postgres:
    image: postgres:16
    ports:
      - "5433:5432"
    volumes:
      - rdsq1-ticket-postgres-data:/var/lib/postgresql/data
      - ./sql:/sql:ro

  ticket-collector:
    image: postgres:16
    depends_on:
      postgres:
        condition: service_healthy
    command: >
      bash -lc "while true; do
        psql -h postgres -U training -d training -f /sql/ticket_collector_tick.sql;
        sleep 10;
      done"
```

Cosa osservare nel Compose

- ▶ nome dei servizi: sono anche nomi DNS nella rete Docker;
- ▶ porte pubblicate: PostgreSQL su 5433, Metabase su 3000;
- ▶ volumi: il database sopravvive al riavvio del container;
- ▶ dipendenze: il collector aspetta che PostgreSQL sia pronto;
- ▶ mount SQL: gli script sono leggibili dal container.

5. Attività pratica

Costruire il primo strato dell'architettura: schema, viste e controlli.

Consegna

La classe deve preparare il database che alimenterà una dashboard ticketing. Prima di pensare ai grafici, bisogna definire quali tabelle e viste rendono il dato affidabile e interrogabile.

- ▶ input: `script sql/ticket_architecture_schema.sql`;
- ▶ output: schema `ticketing`, viste di dashboard e query di controllo;
- ▶ vincolo: distinguere raw, curated e semantic layer;
- ▶ vincolo: ogni vista deve avere granularità chiara;
- ▶ vincolo: verificare raw event, ticket curati e ticket aperti.

- ❶ avviare PostgreSQL;
- ❷ caricare lo schema ticketing;
- ❸ interrogare le tabelle raw e curated;
- ❹ leggere le viste dashboard_*;
- ❺ spiegare quali query userà Metabase nei blocchi successivi.

Implementazione essenziale

```
-- Da psql, dopo avere avviato PostgreSQL
\i sql/ticket_architecture_schema.sql

SET search_path TO ticketing;

SELECT count(*) AS raw_events
FROM support_tickets_raw;

SELECT ticket_id, opened_at, priority, category, region, status
FROM dashboard_ticket_base
ORDER BY opened_at
LIMIT 10;

SELECT day, opened_tickets, resolved_tickets, backlog_delta
FROM dashboard_daily_flow
ORDER BY day;
```

- ▶ quali dati devono essere raw e quali curati?
- ▶ quali vincoli impediscono dashboard sbagliate?
- ▶ quali viste sono contratti stabili per Metabase?
- ▶ cosa succede se il collector invia due volte lo stesso ticket?
- ▶ cosa cambia se una sorgente nuova entra nell'architettura?

- ▶ un'architettura dati è una sequenza di responsabilità, non solo un insieme di container;
- ▶ il DBMS dà forma, vincoli e semantica al dato raccolto;
- ▶ viste e CTE costruiscono il contratto tra database e dashboard.